# Guidelines For `snake_case` Concept Naming


| | |
|---|---|
| Document #: | P1851R0 |
| Date: | 2019-07-28 |
| Project: | Programming Language C++<br>Library Evolution Working Group |


| | |
|---|---|
| Reply-to: | Jonathan Müller<br><jonathan.mueller@foonathan.net> |


## § 1 Introduction

In Cologne, [P1754R1] was adopted changing the naming convention for concepts from `PascalCase` to `snake_case`. While this arguably creates a more consistent standard library naming style, it also opens up the possibility for naming conflicts between concepts and other entities. To tackle this problem, LEWG created guidelines for naming concepts to minimize name conflicts. This paper refines them and proposes to add them to the upcoming LEWG policy standing document (see [P1655R0]).

Note: throughout this paper, I will use `ConceptName` to refer to the theoretical "concept" without prescribing a given name, and `concept_name` to refer to the concrete C++ `concept` with a name following the strategies.

## § 2 Cologne Renaming Guidelines

In Cologne, LEWG used the following guidelines for renaming all the concepts:

1. Concept names are all `snake_case`.
2. Concept names should not have any consistent prefix/suffix to disambiguate (such as `_type` or `_c`).
3. Concepts are renamed according to their category:
   - *Abstractions*, which are high-level concepts like `ForwardIterator`, `View`, or `Sentinel`, are renamed using very generic nouns.
   - *Capabilities*, which are single requirement concepts like `Swappable` or `Constructible`, are named to adjectives `-ible` or `-able`.
   - Other misc. predicate concepts like `Same` or `CommonType` are given names ending in prepositions.

The naming of abstractions (generic nouns) combined with guidelines 1. and 2. allowed for name conflicts. There were two of those: `Iterator` (there is a deprecated class `std::iterator`) and `View` (there is a namespace `std::view`). They were resolved using creativity (`Iterator` is now `input_or_output_iterator`) and renaming of the conflicts (namespace is now `std::views`).

The naming of capabilities and the misc. predicates is free from name conflicts as no entity in the standard library uses such names. As the majority of current standard library concepts fall into this category, the naming guidelines are good at preventing name conflicts.

# 3 Problems With The Cologne Renaming Guidelines

However, those guidelines are not quite perfect given that they were quickly developed during the meeting.

## 3.1 Problem 1: Misc. Predicate Category And Prepositions

The guideline that misc. predicates should end in a preposition has two problems. First, the “misc. predicate” category is (by design) not well-defined, and second, there are other concepts ending in a preposition like `sentinel_for`. It would be nice if there was a more general guideline that covers those cases as well.

There is, and it has to do with the way the concepts are used. In general, a concept can be used in two places:

1. In a `requires` or concept definition:

```
template <typename T>
  requires swappable<T>
void foo(T);

template <typename T>
 requires same_as<T, int>
void bar(T);
```

2. As a type constraint:

```
template <swappable T>
void foo(T);

template <same_as<int> T>
void foo(T);
```

Note that if the concept is used as a type constraint, the first parameter is omitted. For concepts that only have a single parameter, no angle brackets after the name are necessary

at all. Also note that `requires same_as<T, int>` reads a bit weird, whereas `same_as<int> T` is completely natural. From that, the following guideline follows:

> If a concept is mainly used as a type constraint and requires more than one argument, the name should end in a preposition. If a concept is often used in `requires` or in the definition of another concept, the name should not end in a preposition.

This explains all the “misc. predicate” names and also `sentinel_for` and also prevents name conflicts. But it is not completely followed by the current concept names; these ones have multiple required arguments and do not end in a preposition:

- `(regular_)invocable<F, Args...>`
- `predicate<F, Args...>`
- `relation<R, T, U>`
- `strict_weak_order<R, T, U>`
- `writable<Out, T>`
- `output_iterator<I, T>`
- `indirectly_(regular_)unary_invocable<F, I>`
- `indirect_unary_predicate<F, I>`
- `indirect_relation<F, I1>`
- `indirect_streak_weak_order<F, I1>`
- `indirectly_moveable(_storeable)<In, Out>`
- `indirectly_copyable(_storeable)<In, Out>`
- `indirectly_comparable<I1, I2, R>`
- `mergeable<I1, I2, Out>`

Of those concepts, `mergeable` is always used as `requires` and not as type constraint, but I do not know about the other ones. If this revised guidelines is adopted, the ones that are mainly used as type constraints could still be renamed to follow it. But the guideline could simply be that - a guideline, and those concepts reasonable exceptions of the guideline.

### § 3.1.1 Problem 2: How to Deal with Abstraction Conflicts?

Name conflicts between abstraction concepts and types is still a possibility. While there are currently not many of them, this can change as new abstractions are developed (e.g. executors).

Solving the conflicts depends on the situation:

1. A type-erased wrapper is added for a concept.

Then the existing LEWG guideline of using an `any_` prefix applies.

2. A new type is added as an implementation of an existing concept.

   Then the name of the type should encode what is being special about the type compared to the generic concept.

3. A concept is added together with a default implementation for the majority of use cases.

   Even though the majority of use case use the type as the default, the existence of a concept still encourages generic programming: Functions should either be templated or using type erasure. So the name of the concept should be the better and shorter name, as it will be used in the actual applications.

   If the default has special properties, they should be encoded in the name like in the case above. If its only defining property is that it is the default, the name can be just that - `default_<concept>`.

   For example, suppose `std::allocator` did not exist and it is to be added together with an `Allocator` concept. As code will be written in terms of the concept, the name of the concept should be `std::allocator`. The default type can be `std::new_allocator` (naming it after the special property) or `std::default_allocator` (as it is the default).

4. A concept is created to allow replacement of a fixed type in a function.

   For example, suppose there is code that uses `std::string` which should be generalized into a concept. Then the ideal name `string` is already taken, so the concept needs a different name.

   However, this is the wrong approach for designing concepts: the concept requirements should be gathered from the function, not from the type. Doing that will usually also lead to a better name that covers the exact set of requirements.

5. An existing named requirement is conceptified, like creating a concept for the `Allocator` requirement.

   With the new naming scheme, this simply cannot be done in all situations, as the good name is often taken (there is the `std::allocator` type).

There is no good solution for 4. and 5., but 4. is a bad idea and we can live in a world where we do not do 5. So with the rules for type-erased wrappers and `default_<concept>`, as well as the guideline that the concept name is the better and shorter name, there is a guideline that handles conflicts.

# § 4 Proposal

Note that it is quite difficult to provide wording changes to a document that does not exist yet. I have no idea how the style of the document is supposed to be.

Add a new section to the upcoming policy document “Naming of Concepts”:

1 Concept names are:

(1.1) — `snake_case`,

(1.2) — without a suffix or prefix designating them as concepts,

(1.3) — and following the name of a type trait (without the `is_` prefix) if applicable.

2 If a concept is mainly used as a type constraint and has multiple required arguments, the name should end in a suitable preposition. If however the concept is often used in a `requires` clause or concept definition, it should not end in a preposition.

For example, `swappable` requires only a single argument, so it does not end in a preposition, but `swappable_with` requires an additional argument, so it does. Likewise, it is `same_as<int> T`, `constructible_with<args> T` and `sentinel_for<iterator> S`. But `mergeable`, which also requires multiple arguments, does not have a preposition as it is often used in a `requires` clause.

3 The name of a type-erased wrapper of a concept is `any_<concept>`.

For example, it is `any_invocable` instead of `unique_function`.

Add a subsection of “Naming of Concepts”, “Naming of Capability Concepts”:

1 A capability is a concept that has a single requirement, usually a (member) function.

2 The name of a capability concept is an adjective describing the requirement.

3 If the capability is a function, the concept name is formed by taken the function name and using the `-ible` or `-able` suffix.

For example, `swappable` requires `swap()`; `copy_constructible` requires a copy constructor.

Add a subsection of “Naming of Concepts”, “Naming of Abstraction Concepts”:

1 An abstraction is a high-level concept with often multiple requirements or the root of a concept hierarchy.

2 The name of an abstraction concept is a noun introducing new terminology; unlike a capability it does not describe the requirement verbatim.

For example, it is `forward_iterator` not `has_increment_equality_and_dereference`.

3 The name of an abstraction concept is more general than the name of the types satisfying it.

4 If there is a type whose only purpose is to provide a default implementation of an abstraction concept, it can be named `default_<concept>`.

# § 5 References


[P1655R0] Zach Laine. 2019. LEWG Omnibus Design Policy Paper.
https://wg21.link/p1655r0

[P1754R1] Herb Sutter, Casey Carter, Gabriel Dos Reis, Eric Niebler, Bjarne Stroustrup, Andrew Sutton, Ville Voutilainen. 2019. Rename concepts to standard_case for C++20, while we still can.
https://wg21.link/p1754r1